

```
pthread_attr_t gattr;

/* Get the attribute of this thread */
pthread_getattr_np(pthread_self(), &gattr);

CPU_ZERO(&mymask);

/* Get the affinity of the this thread */
pthread_attr_getaffinity_np(&gattr, len, &mymask);

printf("thread_aff: My Inherited Thread Affinity is %p\n", mymask);

CPU_ZERO(&mymask);
/* The 1st Processor in the system */
CPU_SET(0, &mymask);

/* Set the affinity mask for this thread */
if(0 != pthread_attr_setaffinity_np(&gattr, len, &mymask)) {
    perror("thread_aff: Error in pthread_attr_setaffinity_np()");
}

CPU_ZERO(&mymask);
/* Get the affinity of the this thread */
pthread_attr_getaffinity_np(&gattr, len, &mymask);

printf("thread_aff: My Overriden Thread Affinity is %p\n", mymask);

pthread_exit(NULL);
}

int main()
{
    pthread_t pth;
    cpu_set_t mymask;
    unsigned int len = sizeof(mymask);
    unsigned int pid = 0; /* Current Process */

    CPU_ZERO(&mymask);
    /* The 2nd processor in the system */
    CPU_SET(1, &mymask);

    /* Set affinity mask of the process */
    if (sched_setaffinity(pid, len, &mymask) < 0) {
        perror("main: Error in sched_setaffinity()");
        return;
    }

    CPU_ZERO(&mymask);

    /* Get affinity mask of the process */
    sched_getaffinity(pid, len, &mymask);

    printf("main: My Process Affinity is %p\n", mymask);

    pthread_create(&pth, NULL, &thread_aff, NULL);
}
```

```
pthread_join(pth, NULL);

CPU_ZERO(&mymask);
sched_getaffinity(pid, len, &mymask);
printf("Main: My Process Affinity is Still %p\n", mymask);

return 0;
}
```

Note that in order to compile the above code, you need to link it with *pthread* libraries using *-pthread*. To compile it, issue the following command:

```
$ gcc <Program File Name> -o <Output Executable Name> -pthread
```

The above program binds the current process to the second processor in the system, and then creates a thread: *thread_aff*. The *thread_aff*, by default, inherits the same affinity that the parent process currently has (i.e., binds itself with the second processor in the system). This thread then overrides this affinity and binds itself with the very first processor in the system. However, as the scope of *thread_aff* ends, the program again verifies the process affinity to see if an alteration in the thread affinity has made any impact on the process affinity.

Ideally, you should observe that the affinity of the process remains unaffected and tied to the second processor in the system.

I, however, am slightly sceptical about the maturity of these APIs, as sometimes, I observed weird results during my experiments with them.

What next?

Well, I've tried to introduce you to the idea of CPU affinity. Now you can play around with these system calls and experiment further. Besides the Linux man pages that are always a source of great help, Google and other search engines also help in searching for the relevant content.

Happy coding! 

References

- <http://technet.microsoft.com/en-us/library/cc938636.aspx>
- <http://littledaemons.wordpress.com/2009/01/22/cpu-affinity-and-taskset/>
- <http://www.cyberciti.biz/tips/setting-processor-affinity-certain-task-or-process.html>
- <http://www.ibm.com/developerworks/linux/library/l-affinity.html>
- http://en.wikipedia.org/wiki/Processor_affinity
- http://www.kernel.org/doc/man-pages/online/pages/man3/pthread_attr_setaffinity_np.3.html
- http://www.kernel.org/doc/man-pages/online/pages/man3/pthread_getattr_np.3.html

By: Vivek Kumar

The author is a system engineer at IBM India, and works for IBM-ISS (Internet Security Systems). He is based in Pune and you can reach him at mail_DOT_vivekkumar_AT_rediffmail_DOT_com.